

1. Objetivo

Manipular una imagen digital de tipo *bitmap* leída desde un archivo.

2. Ejercicio

Convertir una imagen en color a escala de grises. Una imagen en escala de grises tiene el mismo valor o nivel de gris (**Y**) en sus tres canales. Este valor **Y** de cada píxel se puede obtener a partir de su color **RGB** utilizando la siguiente expresión:

$$\mathbf{Y} = 0.299 \times \mathbf{R} + 0.587 \times \mathbf{G} + 0.114 \times \mathbf{B}$$

La práctica debe cumplir los siguientes requisitos:

» El programa principal (*main*) se debe encargar de leer el archivo "**`imagen.bmp`**" y copiar los datos desde la memoria del *host* hasta la memoria del *device*.

» El *kernel* se debe encargar de convertir la imagen a escala de grises.

» Se deben mostrar las propiedades del dispositivo seleccionado, las dimensiones del *kernel* lanzado, el tiempo de ejecución del *kernel* y visualizar la imagen resultante en escala de grises.



Figura 3.1. Imagen en color y su correspondiente versión en escala de grises.

3. Archivos de imagen

En general, cuando hablamos de formatos de imagen nos referimos al formato del archivo que contiene los datos. La disposición típica de los datos en un archivo de imagen (figura 3.2) consiste en una serie de bytes con información de control (cabecera) seguido de los bytes relativos a la imagen propiamente dicha (datos).

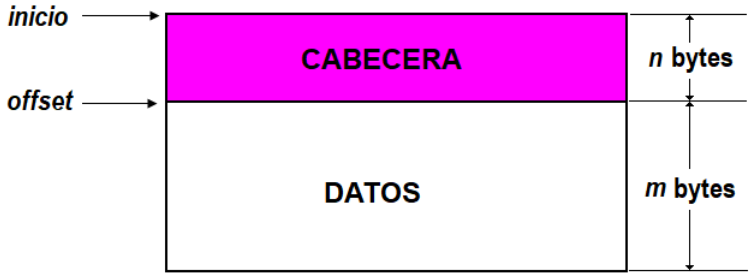


Figura 3.2. Disposición de los datos en un archivo de imagen.

La información de control codificada en la cabecera o *header* del archivo es necesaria para interpretar el bloque con los datos de imagen. Entre otras cosas esta información incluye un identificador del tipo de archivo, las dimensiones de la imagen, la profundidad de color, la ubicación del bloque de datos (*offset*), comentarios, etc.

El bloque correspondiente a los datos de imagen es una codificación del mapa de bits que describe la imagen. Dependiendo del tipo de archivo estos datos pueden estar a su vez comprimidos mediante algún tipo de algoritmo especificado en la cabecera.

3.1. Archivos BMP

El formato de archivo **BMP** es un formato de imagen de mapa de bits propio del sistema operativo Microsoft Windows, en cuyo sistema también se conoce como mapa de bits independiente de dispositivo o *device independent bitmap file format* (**DIB**). Puede albergar imágenes con diferentes profundidades de color (32 bits, 24 bits, 8 bits y 1 bit). En general los datos están sin comprimir (de ahí su gran tamaño) aunque a veces se les puede aplicar una compresión sin pérdida de calidad conocida como compresión **RLE** (Run-length encoding).

Los archivos tipo **BMP** constan de al menos dos cabeceras. La primera de ellas tiene 14 bytes de longitud y en ella se almacena información general del archivo. Esta cabecera comienza con las letras '`BM`' codificadas en ASCII (valores `0x42` y `0x4D`) lo cual permite

identificar el tipo de archivo. La segunda cabecera, conocida como cabecera DIB y generalmente de 40 bytes de longitud, almacena información detallada de la imagen y de la profundidad de color. Al ser un formato típico de los sistemas Windows, el ordenamiento de los datos es de tipo little-endian. Una información completa y detallada del contenido de la cabecera de un fichero **BMP** se muestra en la **TABLA 3**.

TABLA 3. Estructura de la cabecera de un fichero **BMP**.

Dirección	Bytes	Descripción	
0x00	2	Tipo de fichero ("BM").	Bitmap file header
0x02	4	Tamaño del archivo en bytes (incluye cabeceras y datos).	
0x06	2	Reservado.	
0x08	2	Reservado.	
0x0A	4	Dirección de inicio de los datos de la imagen.	
0x0E	4	Tamaño en bytes de la segunda cabecera (40 bytes).	DIB header
0x12	4	Anchura en píxels.	
0x16	4	Altura en píxeles.	
0x1A	2	Número de planos.	
0x1C	2	Número de bits por píxel.	
0x1E	4	Método de compresión utilizado.	
0x22	4	Tamaño de la imagen en bytes.	
0x26	4	Resolución horizontal.	
0x2A	4	Resolución vertical.	
0x2E	4	Tamaño de la tabla de color.	
0x32	4	Contador de colores importantes.	

4. Manejo de archivos

El manejo de archivos en C se realiza a través de una serie de funciones definidas en `<stdio.h>`. Todas ellas consideran que un archivo es una estructura que contiene la información necesaria para su manejo, incluyendo un puntero al buffer de datos, un indicador de posición dentro del buffer y distintos indicadores de estado. Esta estructura se denomina **`FILE`** y la función que crea e inicializa esta estructura para poder manejar un

archivo en particular es **`fopen()`**. Hay varias decenas de funciones relacionadas con el manejo de ficheros (http://www.cplusplus.com/reference/cstdio/) pero las más básicas son las siguientes:

- **`FILE* fopen(const char *filename, const char *mode)`**
  Abre el archivo cuyo nombre está especificado en `filename`. En la apertura en modo binario el parámetro `mode` puede ser "`rb`" para lectura, "`wb`" para escritura o "`ab`" como apéndice. En modo texto[2] no se añade la letra "`b`". Si la función retorna con éxito el archivo se asocia a una estructura `FILE` devuelta como puntero para posteriores referencias.
- **`int fclose(FILE *stream)`**
  Cierra el archivo asociado a la estructura `FILE` devuelta por la función `fopen()`. Si la función retorna con éxito devuelve el valor 0.
- **`size_t fread(void *buf, size_t size, size_t count, FILE *stream)`**
  Lee `count` elementos, cada uno con un tamaño de `size` bytes, desde el fichero asociado a la estructura `stream` y los almacena en el buffer apuntado por `buf`. La función devuelve el número de elementos leídos con éxito.
- **`size_t fwrite(void *buf, size_t size, size_t count, FILE *stream)`**
  Escribe `count` elementos, cada uno con un tamaño de `size` bytes, almacenados en el buffer apuntado por `buf` y los escribe en el fichero asociado a la estructura `stream`. La función devuelve el número de elementos escritos con éxito.
- **`int fseek(FILE *stream, size_t offset, int origin)`**
  Fija el indicador de posición dentro del archivo asociado a la estructura `stream` en una nueva posición definida sumando el valor `offset` a la posición de referencia especificada por `origin`. Esta referencia puede ser el comienzo del archivo (`SEEK_SET`), la posición actual (`SEEK_CUR`) o el final del archivo (`SEEK_END`). Si ha habido éxito la función devuelve el valor 0.
- **`void rewind(FILE *stream)`**
  Fija el indicador de posición del archivo asociado a la estructura `stream` al comienzo del archivo.

4.1. Lectura de un archivo BMP

El siguiente código utiliza una función que se encarga de extraer los datos de un archivo de tipo **BMP**. Dado que la información de color está almacenada en el orden **BGR**, la función se encarga de reordenar los canales y de añadir el canal α para colocar los datos en el orden **RGBA** y así poder utilizar las funciones de **OpenGL** encapsuladas dentro del fichero de cabecera **`gpu_bitmap.h`**:

[2] Los archivos de texto son archivos que contienen caracteres especiales. Dependiendo del sistema donde se esté ejecutando la aplicación se pueden producir conversiones de alguno de estos caracteres especiales durante las operaciones de entrada/salida en modo texto con el fin de adaptarlos al formato de archivo de texto específico del sistema. Aunque hay sistemas que no hacen distinción, utilizar el modo adecuado facilita la portabilidad.

```
////////////////////////////////////////////////////////////////////////
// includes
#include <stdio.h>
#include <stdlib.h>
#include <cuda_runtime.h>
#include "gpu_bitmap.h"

// HOST: funcion llamada desde el host y ejecutada en el host
__host__ void leerBMP_RGBA(const char* nombre, int* w, int* h, unsigned char **imagen);
// Funcion que lee un archivo de tipo BMP:
// -> entrada: nombre del archivo
// <- salida : ancho de la imagen en pixeles
// <- salida : alto de la imagen en pixeles
// <- salida : puntero al array de datos de la imagen en formato RGBA

////////////////////////////////////////////////////////////////////////
// MAIN: rutina principal ejecutada en el host
int main(int argc, char** argv)
{

	////////////////////////////////////////////////////////////////////////
	// Leemos el archivo BMP
	unsigned char *host_color;
	int ancho, alto;

	leerBMP_RGBA("imagen.bmp", &ancho, &alto, &host_color);

	////////////////////////////////////////////////////////////////////////
	// Declaracion del bitmap RGBA:
	// Inicializacion de la estructura RenderGPU
	RenderGPU foto(ancho, alto);

	// Tamaño del bitmap en bytes
	size_t img_size = foto.image_size();

	// Asignacion y reserva de la memoria en el host (framebuffer)
	unsigned char *host_bitmap = foto.get_ptr();

	// Copiamos en el framebuffer los datos leidos para su visualizacion:
	cudaMemcpy(host_bitmap, host_color, img_size, cudaMemcpyHostToHost);

	// Salida
	printf("\n...pulsa [ESC] para finalizar...");

	// Visualizacion
	foto.display_and_exit();

	// Fin
	return 0;
}
////////////////////////////////////////////////////////////////////////
// Funcion que lee un archivo de tipo BMP:
__host__ void leerBMP_RGBA(const char* nombre, int* w, int* h, unsigned char **imagen)
{
	// Lectura del archivo .BMP
	FILE *archivo;

	// Abrimos el archivo en modo solo lectura binario
	if ((archivo = fopen(nombre, "rb")) == NULL)
	{
		printf("\nERROR ABRIENDO EL ARCHIVO %s...", nombre);
		// salida
		printf("\npulsa [INTRO] para finalizar");
		getchar();
		exit(1);
	}
	printf("> Archivo [%s] abierto:\n", nombre);

	// En Windows, la cabecera tiene un tamaño de 54 bytes:
	// 14 bytes (BMP header) + 40 bytes (DIB header)

	// BMP HEADER
	// Extraemos cada campo y lo almacenamos en una variable del tipo adecuado
	// posicion 0x00 -> Tipo de archivo: "BM" (leemos 2 bytes)
	unsigned char tipo[2];
	fread(tipo, 1, 2, archivo);

	// Comprobamos que es un archivo BMP
	if(tipo[0] != 'B' || tipo[1] != 'M' )
	{
		printf("\nERROR: EL ARCHIVO %s NO ES DE TIPO BMP...", nombre);
```

```
        // salida
        printf("\npulsa [INTRO] para finalizar");
        getchar();
        exit(1);
}

// posicion 0x02 -> Tamaño del archivo .bmp (leemos 4 bytes)
unsigned int file_size;
fread(&file_size, 4, 1, archivo);

// posicion 0x06 -> Campo reservado (leemos 2 bytes)
// posicion 0x08 -> Campo reservado (leemos 2 bytes)
unsigned char buffer [4];
fread(buffer, 1, 4, archivo);

// posicion 0x0A -> Offset a los datos de imagen (leemos 4 bytes)
unsigned int offset;
fread(&offset, 4, 1, archivo);

// imprimimos los datos
printf("  \nDatos de la cabecera BMP\n");
printf("> Tipo de archivo      : %c%c\n", tipo[0], tipo[1]);
printf("> Tamano del archivo   : %u KiB\n", file_size / 1024);
printf("> Offset de datos      : %u bytes\n", offset);

// DIB HEADER
// Extraemos cada campo y lo almacenamos en una variable del tipo adecuado
// posicion 0x0E -> Tamaño de la cabecera DIB (BITMAPINFOHEADER) (leemos 4 bytes)
unsigned int header_size;
fread(&header_size, 4, 1, archivo);

// posicion 0x12 -> Ancho de la imagen (leemos 4 bytes)
unsigned int ancho;
fread(&ancho, 4, 1, archivo);

// posicion 0x16 -> Alto de la imagen (leemos 4 bytes)
unsigned int alto;
fread(&alto, 4, 1, archivo);

// posicion 0x1A -> Numero de planos de color (leemos 2 bytes)
unsigned short int planos;
fread(&planos, 2, 1, archivo);

// posicion 0x1C -> Profundidad de color (leemos 2 bytes)
unsigned short int color_depth;
fread(&color_depth, 2, 1, archivo);

// posicion 0x1E -> Tipo de compresion (leemos 4 bytes)
unsigned int compresion;
fread(&compresion, 4, 1, archivo);

// imprimimos los datos
printf("  \nDatos de la cabecera DIB\n");
printf("> Tamano de la cabecera: %u bytes\n", header_size);
printf("> Ancho de la imagen   : %u pixeles\n", ancho);
printf("> Alto de la imagen    : %u pixeles\n", alto);
printf("> Planos de color      : %u\n", planos);
printf("> Profundidad de color : %u bits/pixel\n", color_depth);
printf("> Tipo de compresion   : %s\n", (compresion == 0) ? "none" : "unknown");

// LEEMOS LOS DATOS DEL ARCHIVO
// Calculamos espacio para una imagen de tipo RGBA:
size_t  img_size = ancho * alto * 4;

// Reserva para almacenar los datos del bitmap
unsigned char *datos = (unsigned char*)malloc(img_size);;

// Desplazamos el puntero FILE hasta el comienzo de los datos de imagen: 0 + offset
fseek(archivo, offset, SEEK_SET);

// Leemos pixel a pixel, reordenamos (BGR -> RGB) e insertamos canal alfa
unsigned int pixel_size = color_depth / 8;
for (unsigned int i = 0; i < ancho * alto; i++)
{
        fread(buffer, 1, pixel_size, archivo); // leemos el pixel i
        datos[i * 4 + 0] = buffer[2]; // escribimos canal R
        datos[i * 4 + 1] = buffer[1]; // escribimos canal G
        datos[i * 4 + 2] = buffer[0]; // escribimos canal B
        datos[i * 4 + 3] = buffer[3]; // escribimos canal alfa (si lo hay)
}
```

```
        // Cerramos el archivo
        fclose(archivo);

        // PARAMETROS DE SALIDA
        // Ancho de la imagen en pixeles
        *w = ancho;
        // Alto de la imagen en pixeles
        *h = alto;
        // Puntero al array de datos RGBA
        *imagen = datos;

        // Salida
        return;
}
```